

UNIT II — Data Manipulation using Pandas

Syllabus: Pandas Objects – DataFrames and Series | Data Indexing and Selection (loc, iloc) | Operating on Data (Element-wise and Broadcasting) | Handling Missing Data & Imputation | Hierarchical Indexing and MultiIndex Objects | Combining Datasets (concat, append, merge, join) | Aggregation and Grouping (GroupBy) | Pivot Tables and Cross-tabulation | Data Visualization for EDA (Matplotlib, Seaborn, Plotly)

1. Introduction to Data Manipulation with Pandas

1.1 Why Data Manipulation?

- Machine learning models need **clean data** to train and test on.
- Real-world datasets commonly contain:
 - Missing values (NaN, Null, NA)
 - Irrelevant or redundant rows/columns
- **Data Wrangling** is the process of cleaning and reshaping data so it is ready for use.

1.2 Why Pandas?

- Pandas is a Python library built on top of **NumPy**.
- Two core objects:
 - **Series** – a one-dimensional labeled array
 - **DataFrame** – a two-dimensional, multi-dimensional-labeled table built from Series
- Supports heterogeneous data types and missing values.
- Offers operations similar to spreadsheets and SQL.

1.3 Installing Pandas

```
bash
pip install pandas
```

2. Creating and Exploring a DataFrame

2.1 Creating a DataFrame

```
python
import pandas as pd

student_register = pd.DataFrame()
student_register['Name'] = ['Abhijit', 'Smriti', 'Akash', 'Roshni']
student_register['Age'] = [20, 19, 20, 14]
student_register['Student'] = [False, True, True, False]

print(student_register)
```

Output

```
Name Age Student
```

```
0 Abhijit 20 False
1 Smriti 19 True
2 Akash 20 True
3 Roshni 14 False
```

The index column `[0, 1, 2, 3]` is auto-generated by Pandas and can be customized.

2.2 Adding a Row with `append()`

python

```
new_person = pd.Series(['Mansi', 19, True], index=['Name', 'Age', 'Student'])
student_register = student_register.append(new_person, ignore_index=True)
```

- A DataFrame is essentially a stack of **Series** objects (rows).
- Appending a row = creating a new Series and adding it to the DataFrame.
- (Note: `.append()` is deprecated in modern Pandas — prefer `pd.concat()`.)

2.3 Exploring a Dataset

Function	Purpose
<code>.shape</code>	Returns dimensions (rows, columns)
<code>.info()</code>	Column types, non-null counts, memory usage
<code>.describe()</code>	Statistical summary (count, mean, std, min, max, quartiles)
<code>.corr()</code>	Correlation between numeric columns

python

```
print(student_register.shape)
print(student_register.info())
print(student_register.describe())
```

`.describe()` gives:

- count — total entries
- mean — average value
- std — standard deviation
- min / max — range
- 25% / 50% / 75% — quartiles (useful for understanding distribution in ML)

2.4 Dropping Columns and Rows

python

```
students = student_register.drop('Age', axis=1) # drop column
students = students.drop(2, axis=0)           # drop row
```

3. Pandas Objects: Series and DataFrame

3.1 The Series Object

A **Series** is a one-dimensional array with axis labels (an index). It behaves like a hybrid of a NumPy array and a Python dictionary.

python

```
data = pd.Series([0.25, 0.5, 0.75, 1.0])
data[1]    # 0.5
data[1:3]  # slice
data.values # underlying NumPy array
```

```
data.index    # the Index object
```

Custom index:

```
python
data = pd.Series([0.25, 0.5, 0.75, 1.0], index=['a', 'b', 'c', 'd'])
data['b']    # 0.5
```

Series as a dictionary:

```
python
population_dict = {'California': 38332521, 'Texas': 26448193, 'New York': 19651127}
population = pd.Series(population_dict)
population['California']    # 38332521
population['California':'Illinois'] # slicing by label
```

Ways to construct a Series:

```
python
pd.Series([2, 4, 6])    # from a list
pd.Series(5, index=[100, 200, 300]) # from a scalar
pd.Series({'2': 'a', 1: 'b', 3: 'c'}) # from a dictionary
```

3.2 The DataFrame Object

A **DataFrame** is a collection of aligned Series — a 2D, table-like structure with row and column indices.

```
python
area = pd.Series({'California': 423967, 'Texas': 695662, 'New York': 141297})
states = pd.DataFrame({'population': population, 'area': area})
```

```
states.index    # row labels
states.columns  # column labels
states['area']  # access a column (returns a Series)
```

Ways to construct a DataFrame:

```
python
pd.DataFrame(population, columns=['population'])    # from a Series
pd.DataFrame([{'a': i, 'b': 2*i} for i in range(3)]) # from a list of dicts
pd.DataFrame({'population': population, 'area': area}) # from a dict of Series
pd.DataFrame(np.random.rand(3, 2), columns=['foo', 'bar'], index=['a', 'b', 'c']) # from NumPy array
```

3.3 The Index Object

Index objects are **immutable** arrays used to label rows/columns and support set operations.

```
python
ind = pd.Index([2, 3, 5, 7, 11])
ind[1]    # 3
ind[::2]  # [2, 5, 11]
ind.size, ind.shape, ind.ndim, ind.dtype

indA = pd.Index([1, 3, 5, 7, 9])
indB = pd.Index([2, 3, 5, 7, 11])
indA & indB    # intersection → [3, 5, 7]
indA | indB    # union → [1, 2, 3, 5, 7, 9, 11]
indA ^ indB    # symmetric difference → [1, 2, 9, 11]
```

4. Data Indexing and Selection

Pandas indexing extends NumPy-style indexing (`arr[2,1]`, slicing, masking, fancy indexing) with label-aware features.

4.1 Selection in a Series

As a dictionary:

```
python
data = pd.Series([0.25, 0.5, 0.75, 1.0], index=['a', 'b', 'c', 'd'])
data['b']          # 0.5
'a' in data        # True
data.keys()
list(data.items())
data['e'] = 1.25    # Series are mutable, like dicts
```

As a one-dimensional array:

```
python
data['a':'c']      # explicit slice — 'c' is INCLUDED
data[0:2]          # implicit slice — index 2 EXCLUDED
data[(data > 0.3) & (data < 0.8)] # masking
data[['a', 'e']]   # fancy indexing
```

4.2 `loc`, `iloc` (and the deprecated `ix`)

When a Series has an **integer index**, `data[1]` and `data[1:3]` can be ambiguous — is it label-based or position-based?

```
python
data = pd.Series(['a', 'b', 'c'], index=[1, 3, 5])

data.loc[1]        # explicit/label index → 'a'
data.loc[1:3]      # label-based, inclusive of both ends

data.iloc[1]       # implicit/position index → 'b'
data.iloc[1:3]     # position-based, exclusive of end
```

Indexer	Basis	Notes
<code>loc</code>	Explicit (label) index	Inclusive of end in slices
<code>iloc</code>	Implicit (position) index	Exclusive of end, like Python lists
<code>ix</code>	Hybrid of both	Deprecated — avoid

Best practice: always prefer `loc/iloc` over plain `[]` for clarity and to avoid bugs.

4.3 Selection in a DataFrame

A DataFrame behaves both as a **dictionary of Series** and as a **2D array**.

As a dictionary of columns:

```
python
data['area']        # dictionary-style
data.area           # attribute-style (fails if column name isn't a valid identifier or clashes with a method, e.g. 'pop')
data['density'] = data['pop'] / data['area'] # add a new column
```

As a 2D array:

```
python
data.values      # raw NumPy array
data.T           # transpose

data.iloc[:3, :2]      # implicit indexing
data.loc[:, 'Illinois', :pop]  # explicit indexing
data.loc[data.density > 100, ['pop', 'density']] # boolean mask + fancy indexing
data.iloc[0, 2] = 90    # setting a value
```

Row-focused shortcuts:

```
python
data['Florida': 'Illinois'] # slice by row label
data[1:3]                   # slice by row position
data[data.density > 100]    # row-wise boolean mask
```

5. Operating on Data: Element-wise Operations & Broadcasting

Index Alignment

When performing binary operations, Pandas **aligns indices automatically**; unmatched labels produce NaN.

```
python
area = pd.Series({'Alaska': 1723337, 'Texas': 695662, 'California': 423967})
pop = pd.Series({'California': 38332521, 'Texas': 26448193, 'New York': 19651127})
print(pop / area)          # NaN where labels don't match

pop.add(area, fill_value=0) # control the fill value instead of NaN
```

This also applies to DataFrames, aligning **both rows and columns**.

Output:

```
Alaska      NaN
California   90.413
Texas       38.019
New York     NaN
dtype: float64
```

Operator ↔ Method mapping:

Operator	Method
+	<code>add()</code>
-	<code>sub()</code> , <code>subtract()</code>
*	<code>mul()</code> , <code>multiply()</code>
/	<code>truediv()</code> , <code>divide()</code>
//	<code>floordiv()</code>
%	<code>mod()</code>
**	<code>pow()</code>

Element-wise Operations vs. Broadcasting

- **Element-wise:** both arrays must be the **same shape**; operation applied position by position.

Example :

```
a = np.array([1, 2, 3]);
b = np.array([4, 5, 6])
a + b # [5, 7, 9]
```

- **Broadcasting:** the smaller array is automatically "stretched" to match the larger array's shape.

Example 1:

```
a = np.array([[1, 2, 3], [4, 5, 6]])
b = np.array([10, 20, 30])
a + b # [[11, 22, 33], [14, 25, 36]]
```

Example 2:

```
A = np.array([1, 2, 3])
B = 2

print(A * B)
```

Output:

```
[2 4 6]
```

6. Handling Missing Data

6.1 What Counts as Missing Data?

- Occurs when no value is recorded for an item.
- Pandas refers to missing data generically as **NA (Not Available)**.
- Represented by:
 - **None** — Python's singleton object; only usable in `object-dtype` arrays.
 - **NaN** — "Not a Number," an IEEE floating-point value; preferred for numeric data because it supports fast vectorized computation.

6.2 Detecting Missing Values

Functions:

```
df.isna()    # Returns True where data is missing
df.isnull()  # Same as isna()
df.notnull() # Returns True where data is present
```

```
df[df.isnull().any(axis=1)]
→ Displays rows with at least one missing value
```

`df.isnull().sum()`
→ Counts missing values column-wise

6.3 Handling Missing Values

Function	Purpose
<code>isnull()</code>	Detect missing values
<code>notnull()</code>	Opposite of <code>isnull()</code>
<code>dropna()</code>	Remove rows/columns containing missing data
<code>fillna(value)</code>	Replace <code>NaN</code> with a specified value
<code>replace()</code>	Replace specific values (not limited to <code>NaN</code>) using a value, regex, list, or dict
<code>interpolate()</code>	Estimate and fill missing values using interpolation

7. Hierarchical Indexing (MultiIndex)

7.1 What Is a MultiIndex?

- A **hierarchical index** allows more than one level of row/column labeling — useful when data has more than two logical dimensions.
- Structure: Series (1D) → DataFrame (2D) → MultiIndex DataFrame (3D+).

7.2 Creating a MultiIndex

Example:

```
index = pd.MultiIndex.from_tuples([('2023', 'Jan'), ('2023', 'Feb'), ('2024', 'Jan')], names=['Year', 'Month'])
df = pd.DataFrame({'Sales': [1000, 1200, 1300], 'Profit': [200, 250, 300]}, index=index)
```

Output:

		Sales	Profit
Year	Month		
2023	Jan	1000	200
	Feb	1200	250
2024	Jan	1300	300

7.3 Selecting from a MultiIndex

```
df.loc['2023']          # all rows for year 2023
df.loc[('2023', 'Jan')] # a specific row
```

7.4 Reorganizing a MultiIndex

Operation	Purpose
<code>swaplevel(0, 1)</code>	Swap the position of two index levels
<code>sort_index(level=1)</code>	Sort based on a given index level
<code>.sum(level='exam')</code>	Aggregate statistics per index level
<code>set_index(['a', 'b'])</code>	Move columns into the row index
<code>reset_index()</code>	Move index levels back into columns

7.5 MultiIndex in Columns

```
arrays = [['Sales', 'Sales', 'Profit', 'Profit'], ['Q1', 'Q2', 'Q1', 'Q2']]
column_index = pd.MultiIndex.from_arrays(arrays, names=['Metric', 'Quarter'])
df = pd.DataFrame([[100, 150, 20, 25], [200, 250, 30, 35]], columns=column_index)
```

Output:

Metric	Sales		Profit	
Quarter	Q1	Q2	Q1	Q2
0	100	150	20	25
1	200	250	30	35

Reshaping operations:

- `stack()` — converts a column level into an index level
- `unstack()` — converts an index level into a column level
- `reset_index()` — flattens a MultiIndex

8. Combining Datasets

8.1 `concat()` — Stacking DataFrames

Usage	Description
<code>pd.concat([df1, df2])</code>	Vertical stacking (adds rows)
<code>pd.concat([df1, df2], axis=1)</code>	Horizontal stacking (adds columns)
Different indexes	Aligns by index; missing values filled with NaN
<code>pd.concat([df1, df2], keys=['df1', 'df2'])</code>	Adds a hierarchical (MultiIndex) grouping
python	
<code>pd.concat([df1, df2])</code>	# vertical
<code>pd.concat([df1, df2], axis=1)</code>	# horizontal
<code>pd.concat([df3, df4], axis=1)</code>	# different indexes → NaN for gaps
<code>pd.concat([df1, df2], keys=['df1', 'df2'])</code>	# MultiIndex result

Example :

```
import pandas as pd

df1 = pd.DataFrame({'A': [1, 2], 'B': [3, 4]})
df2 = pd.DataFrame({'A': [5, 6], 'B': [7, 8]})

pd.concat([df1, df2])      // Vertical stacking
```

Output:

	A	B
0	1	3
1	2	4
0	5	7
1	6	8

Horizontal Stacking:


```
pd.concat([df1, df2], axis=1)
```

Output:

	A	B	A	B
0	1	3	5	7
1	2	4	6	8

Different Index → NaN:

```
df3 = pd.DataFrame({'A': [1, 2]}, index=[0,1])
df4 = pd.DataFrame({'B': [3, 4]}, index=[1,2])

pd.concat([df3, df4], axis=1)
```

Output:

	A	B
0	1.0	NaN
1	2.0	3.0
2	NaN	4.0

Using keys (MultiIndex):

```
pd.concat([df1, df2], keys=['df1', 'df2'])
```

Output:

		A	B
df1	0	1	3
	1	2	4
df2	0	5	7
	1	6	8

8.2 `append()` — Adding Rows

Returns a **new** DataFrame; the originals are unchanged unless reassigned.

Example:

```
df1.append(df2, ignore_index=True) # simple row append
df3.append(df4, ignore_index=True) # mismatched columns → NaN filled in automatically
df5.append(pd.Series({'A': 5, 'B': 6}), ignore_index=True) # append a single row (Series)
```

Output:

	A	B
0	1	3
1	2	4
2	5	7
3	6	8

Output:

	A	B
0	1	3
1	2	4
2	5	6

8.3 `merge()` — Combining by Column Values

Example:

```
import pandas as pd

df1 = pd.DataFrame({'key': ['A', 'B'], 'val1': [1, 2]})
df2 = pd.DataFrame({'key': ['A', 'C'], 'val2': [3, 4]})
```

Output of df1

```
   key  val1
0    A     1
1    B     2
```

Output of df2

```
   key  val2
0    A     3
1    C     4
```

Example:

```
pd.merge(df1, df2, on='key', how='inner') # only matching keys

pd.merge(df1, df2, on='key', how='left')  # all from left, matched from right
pd.merge(df1, df2, on='key', how='outer') # all rows from both, NaN where unmatched
```

Inner Join

```
pd.merge(df1, df2, on='key', how='inner')
```

Output:

```
   key  val1  val2
0    A     1     3
```

Left Join

```
pd.merge(df1, df2, on='key', how='left')
```

Output:

```
   key  val1  val2
0    A     1   3.0
1    B     2   NaN
```

Outer Join

```
pd.merge(df1, df2, on='key', how='outer')
```

Output

```
   key  val1  val2
0    A   1.0   3.0
1    B   2.0   NaN
2    C   NaN   4.0
```

8.4 join() — Combining by Index

join() combines DataFrames by matching their index values, similar to a database join but performed on row indexes instead of columns.

Example:

```
import pandas as pd

df3 = pd.DataFrame({'A': [1, 2]}, index=['x', 'y'])
df4 = pd.DataFrame({'B': [3, 4]}, index=['y', 'z'])
```

Output of df3:

```
   A
x  1
y  2
```

Output of df4:

```
   B
y  3
z  4
```

Example:

```
df3.join(df4, how='inner') # only matching index labels
df3.join(df4, how='left')  # all rows from df3, matched from df4
```

Inner:

```
df3.join(df4, how='inner')
```

Output:

```
   A  B
y  2  3
```

Left:

```
df3.join(df4, how='left')
```

Output:

```
   A    B
x  1  NaN
y  2  3.0
```

9. Aggregation and Grouping

Aggregation:

Aggregation is the process of applying one or more functions to a dataset to compute summary statistics.

Functions are,

- `sum()`
- `mean()`
- `count()`
- `min()`
- `max()`

Grouping:

Grouping is the process of dividing data into groups based on specific column values to perform analysis on each group.

9.1 groupby () :

Example:

```
import pandas as pd
```

```
df = pd.DataFrame({'Category': ['A','A','B','B','C'], 'Value': [10,15,10,20,25]})
```

```
grouped = df.groupby('Category')
```

```
grouped.sum() # aggregate with sum
```

```
grouped.agg(['sum', 'mean', 'count']) # multiple aggregations at once
```

```
def range_func(x):
```

```
    return x.max() - x.min()
```

```
grouped.agg(range_func) # custom aggregation function
```

Output:

Category	Value
A	25
B	30
C	25

Category	sum	mean	count
A	25	12.5	2
B	30	15.0	2
C	25	25.0	1

Category	Value
A	5
B	10
C	0

9.2 Grouping by Multiple Columns

Example 1:

```
import pandas as pd
```

```
df_multi = pd.DataFrame({
```

```
    'Category': ['A','A','B','B','C'],
```

```
    'SubCategory': ['X','Y','X','Y','X'],
```

```
    'Value': [10,15,10,20,25]
```

```
})
```

```
print(df_multi.groupby(['Category', 'SubCategory']).sum())
```

Output:

Category	SubCategory	Value
A	X	10
	Y	15
B	X	10
	Y	20
C	X	25

Example 2:

```
import pandas as pd

df = pd.DataFrame({
    'Department': ['IT', 'IT', 'IT', 'HR', 'HR', 'Sales', 'Sales', 'Sales'],
    'Gender': ['M', 'M', 'F', 'M', 'F', 'M', 'F', 'F'],
    'Salary': [50000, 55000, 60000, 45000, 48000, 55000, 52000, 53000]
})

print(df)
```

Input:

	Department	Gender	Salary
0	IT	M	50000
1	IT	M	55000
2	IT	F	60000
3	HR	M	45000
4	HR	F	48000
5	Sales	M	55000
6	Sales	F	52000
7	Sales	F	53000

Grouping with sum():

```
print(df.groupby(['Department', 'Gender']).sum())
```

Output:

	Department	Gender	Salary
	HR	F	48000
		M	45000
	IT	F	60000
		M	105000
	Sales	F	105000
		M	55000

10. Pivot Tables and Cross-tabulation

Pivot Table:

A Pivot Table in Pandas is a powerful data summarization tool used to reorganize, group, and aggregate data from a dataset. It allows you to transform rows into columns (or vice versa) and compute summary statistics like sum, mean, count, etc.

10.1 Creating a Pivot Table

```
pd.pivot_table(df_multi, values='Value', index=['Category'], columns=['SubCategory'], aggfunc='sum')
```

10.2 Full Syntax

```
pd.pivot_table(
    df_multi, values=None, index=None, columns=None,
    aggfunc='mean', fill_value=None, margins=False,
    dropna=True, margins_name='All', observed=False
)
```

Input:

Category	SubCategory	Value
A	X	10
A	Y	15
B	X	10
B	Y	20
C	X	25

Output :

SubCategory	X	Y
Category		
A	10	15
B	10	20
C	25	NaN

Example :

```
import pandas as pd

pd.pivot_table(
    df_multi,
    values='Value',
    index='Category',
    columns='SubCategory',
    aggfunc='sum',
    fill_value=0,
    margins=True,
    margins_name='Total'
)
```

10.3 Key Parameters

Parameter	Description
df_multi	The DataFrame to pivot
values	Column(s) to aggregate
index	Row grouping labels
columns	Column grouping labels
aggfunc	Aggregation function(s): string, NumPy function, or dict
fill_value	Value used to replace missing entries
margins	Adds subtotal row/column when <code>True</code>
margins_name	Custom label for the totals row/column
dropna	Whether to drop groups that are entirely NaN
observed	How categorical data is handled

10.4 Filtering a Pivot Table

```
pivot_df[pivot_df['Sales'] > 5000] # constant threshold
pivot_df[pivot_df['Sales'] > pivot_df['Sales'].mean()] # dynamic threshold
```

11. String Operations

The `.str` accessor applies Python string methods **element-wise** across an entire Series, automatically skipping missing values — no explicit loops required.

11.1 Common .str Methods

Operation	Method
Lowercase	<code>str.lower()</code>
Uppercase	<code>str.upper()</code>
String length	<code>str.len()</code>
Check substring	<code>str.contains()</code>
Replace substring	<code>str.replace()</code>
Split / join	<code>str.split()</code> , <code>str.join()</code>
Remove whitespace	<code>str.strip()</code>
Extract substring	<code>str[:N]</code>
Concatenate strings	<code>+ operator</code>
Count / find	<code>str.count()</code> , <code>str.find()</code> , <code>str.findall()</code>
Start/end check	<code>str.startswith()</code> , <code>str.endswith()</code>
Case check	<code>str.islower()</code> , <code>str.isupper()</code> , <code>str.isnumeric()</code>
Swap case	<code>str.swapcase()</code>
Repeat	<code>str.repeat(n)</code>

11.2 Examples

python

```
df = pd.DataFrame({'Text': ['Hello', 'World', 'Pandas']})
```

```
df['Text_Lower'] = df['Text'].str.lower()
df['Text_Upper'] = df['Text'].str.upper()
df['Text_Length'] = df['Text'].str.len()
df['Contains_o'] = df['Text'].str.contains('o')
df['Text_Replaced'] = df['Text'].str.replace('o', 'O')
df['Text_Split'] = df['Text'].str.split()
df['Text_Substr'] = df['Text'].str[:3]
```

```
df_ws = pd.DataFrame({'Text': [' Hello ', ' World ']}).
```

```
df_ws['Text_Strip'] = df_ws['Text'].str.strip()
```

```
df['Text_Concat'] = df['Text'] + ' ' + df['More_Text']
```

Cross Tabulation :

A **Cross Tabulation (Crosstab)** in Pandas is a method used to **summarize the relationship between two or more categorical variables** by displaying their **frequency (count) in a table format**.

→ **Crosstab shows how many times combinations of categories occur.**

Syntax:

```
pd.crosstab(index, columns)
```

Example:

Input:

```
import pandas as pd
```

```
df = pd.DataFrame({
```

```

        'Department': ['IT', 'IT', 'HR', 'HR', 'Sales', 'Sales', 'IT'],
        'Gender': ['M', 'F', 'M', 'F', 'M', 'F', 'M']
    })

print(df)

```

Crosstab Operation

```
pd.crosstab(df['Department'], df['Gender'])
```

Output:

Gender	F	M
Department		
HR	1	1
IT	1	2
Sales	1	1

To get Total:

```
pd.crosstab(df['Department'], df['Gender'], margins=True)
```

Output:

Gender	F	M	All
Department			
HR	1	1	2
IT	1	2	3
Sales	1	1	2
All	3	4	7

12. Data Visualization for EDA

→ Data Visualization in EDA is the process of using graphs and charts to understand patterns, trends, distributions, and relationships in data.

It helps in:

- Identifying patterns
- Detecting outliers
- Understanding distributions
- Making decisions based on data

→ Pandas integrates with visualization libraries to support exploratory data analysis (EDA):

1. Matplotlib

→ **Matplotlib** is a **basic and powerful plotting library** used to create static graphs.

Features:

- Foundation library for plotting
- Highly customizable
- Supports line, bar, scatter, histogram

Example:


```
import matplotlib.pyplot as plt

x = [1,2,3,4]
y = [10,20,15,25]

plt.plot(x, y)
plt.title("Line Chart")
plt.show()
```

2.Seaborn

→**Seaborn** is built on Matplotlib and provides **advanced statistical visualizations** with better design.

Features:

- Attractive default styles
- Works well with Pandas DataFrames
- Built-in statistical plots

Example:

```
import seaborn as sns
import pandas as pd

df = pd.DataFrame({'A': [1,2,3,4], 'B': [10,20,15,25]})

sns.barplot(x='A', y='B', data=df)
```

3. Plotly

- **Plotly** is a library used to create **interactive and dynamic visualizations**.

Features:

- Interactive charts (zoom, hover)
- Used in dashboards
- Web-based visualization

Example:

```
import plotly.express as px

df = px.data.iris()
fig = px.scatter(df, x='sepal_width', y='sepal_length')
fig.show()
```

Pandas integrates with visualization libraries to support exploratory data analysis (EDA):

13. Key Takeaways:

- **Pandas** builds on NumPy to provide labeled, flexible **Series** and **DataFrame** structures.

- `loc/iloc` give unambiguous label-based vs. position-based indexing.
- **Broadcasting** and **index alignment** let Pandas operate cleanly on differently-shaped or differently-labeled data.
- **Missing data** (`NaN/None`) can be detected (`isnull`), removed (`dropna`), or filled (`fillna`, `interpolate`).
- **MultiIndex** enables higher-dimensional, hierarchical data representation.
- `concat`, `append`, `merge`, `join` each combine datasets in different ways — by axis, by column key, or by index.
- `groupby` and `pivot_table` are the two main tools for aggregation and summarization.
- `.str` accessor enables efficient, vectorized text cleaning.
- **Matplotlib, Seaborn, and Plotly** extend Pandas for visual exploratory data analysis.